

Start where you are. Use what you have. Do what you can

Object Oriented PROGRAMMING



ENCAPSULATION

INHERITANCE

ABSTRACTION

POLYM.

Inheritance

The capability of a class to derive properties and characteristics from another class is called Inheritance.

```
class Person
{
    age
    name
    lang.
}
```

```
class Yash → Person
{
    age
    name
    lang
    habits
}
```

parent
Super

Base Class

child
Subclass

Derived Class

Syntax Inheritance

C++

```
// Base Class
class Person{

};

// Derived Class
class Pramod: public Person{

};
```

JAVA

```
// Base Class
class Person{ 3 usages 1 inheritance

}

// Derived Class
class Pramod extends Person{ 2 usages

}
```

```
// Base Class
```

```
class A{
```

```
};
```

```
// Base class
```

```
class B{
```

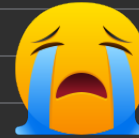
```
}
```

```
// Derived Class
```

```
class C: public A, public B{
```

```
};
```

In java, we cannot extend more than one class



Mode of Inheritance

Public mode: If we derive a sub class from a public base class. Then the public member of the base class will become public in the derived class and protected members of the base class will become protected in derived class.

Protected mode: If we derive a sub class from a Protected base class. Then both public member and protected members of the base class will become protected in derived class.

Private mode: If we derive a sub class from a Private base class. Then both public member and protected members of the base class will become Private in derived class.

Base class member access specifier	Type of Inheritance		
	Public	Protected	Private
Public	Public	Protected	Private
Protected	Protected	Protected	Private
Private	Not accessible (Hidden)	Not accessible (Hidden)	Not accessible (Hidden)

```

class A // Base
{
    private
        int x

```

```

}

// Derived
class B: private A
{
    ?
    int x
}

```

Base class	mode of inherit		
	public	protected	private
public	public	protected	private
protected	protected	protected	private
private	NA	NA	NA

```
class A
{
    protected:
        int x
        String y
}
```

```
class B: public A
{
    protected
        x
        y
}
```

B obj

Types of Inheritance

Single Inheritance

Multilevel Inheritance

Multiple Inheritance

Hierarchical Inheritance

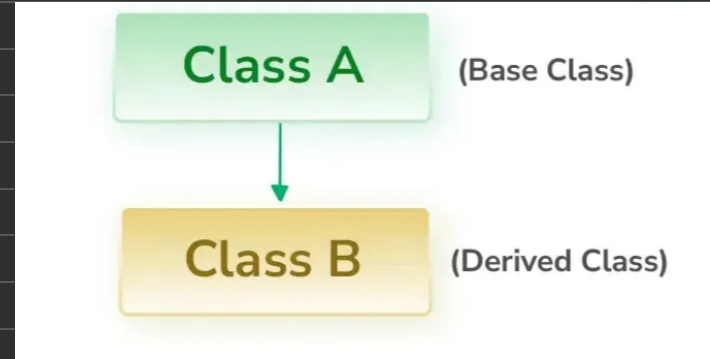
Single Inheritance

In single inheritance, a class is allowed to inherit from only one class. i.e. one base class is inherited by one derived class only.

```
class Mango  
{
```

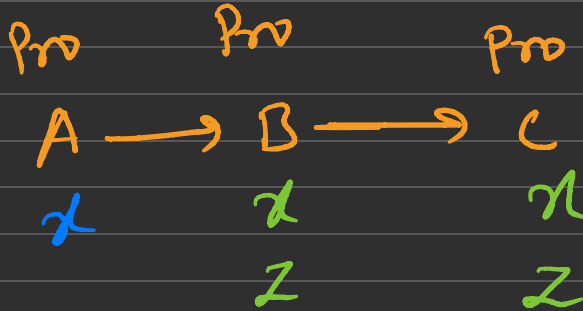
```
{
```

```
class Alphonso : protected Mango  
{
```



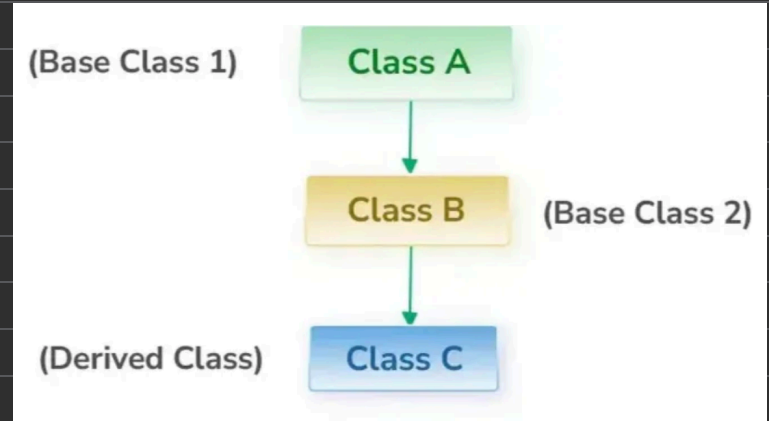
Multilevel Inheritance

A derived class is created from another derived class and that derived class can be derived from a base class or any other derived class. There can be any number of levels.



$$A = B \quad B = C$$

$$A = C$$



```
class Animal  
{  
    string type
```

```
}
```

```
class Dog : protected Animal  
{  
    string sound
```

```
}
```

```
class GoldenRhiner : protected Dog  
{  
    int weight
```

Multiple Inheritance

Multiple Inheritance where a class can inherit from more than one class. i.e one Derived is inherited from more than one Base Class.

```
class A
```

```
{
```

```
}
```

```
class B
```

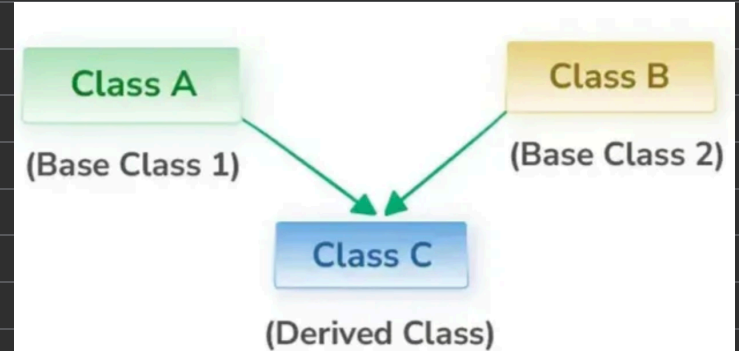
```
{
```

```
}
```

```
class C : A, B
```

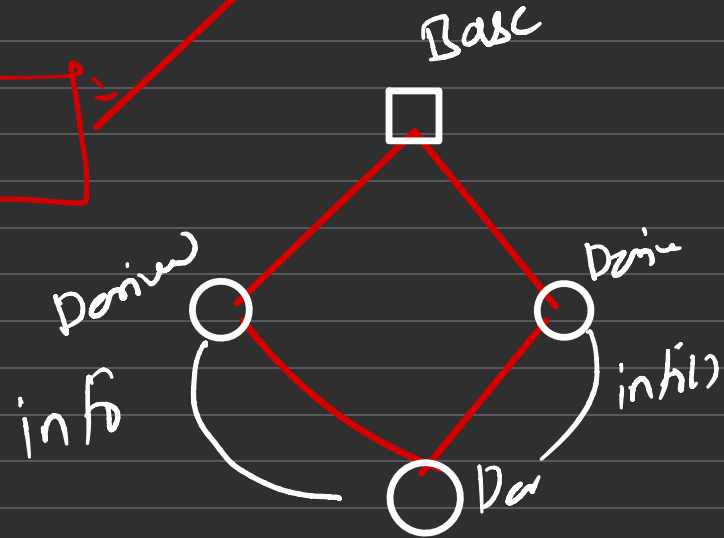
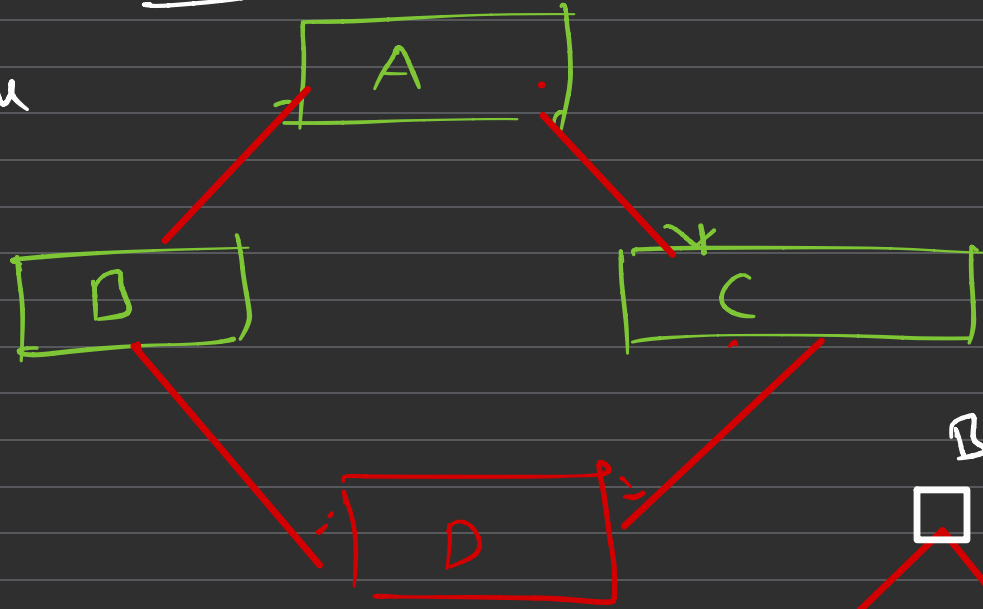
```
{
```

```
}
```



Virtual X — Saved

Unique Name



Hierarchical Inheritance

more than one subclass is inherited from a single base class. i.e. more than one derived class is created from a single base class.

```
class Person
```

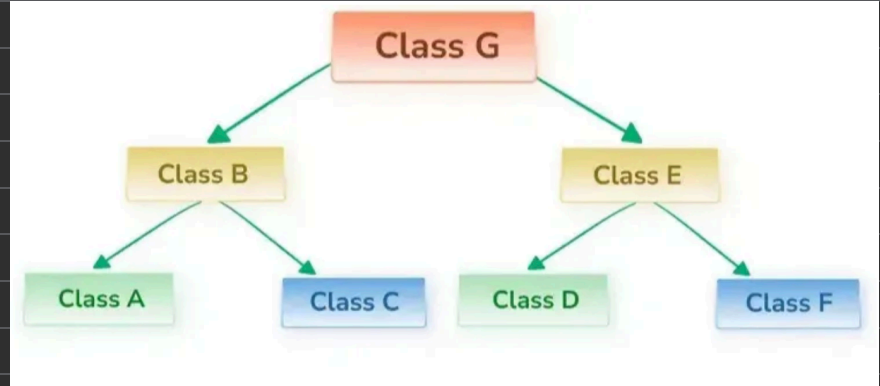
```
{
```

```
{
```

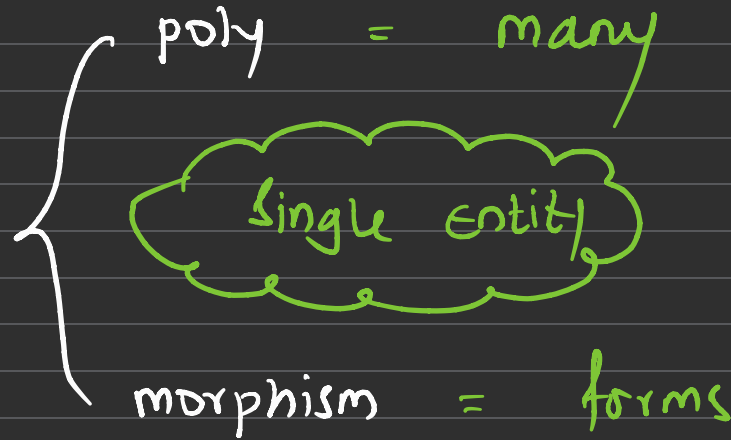
```
class Harsh inherit Person
```

```
class Yash inherit Person
```

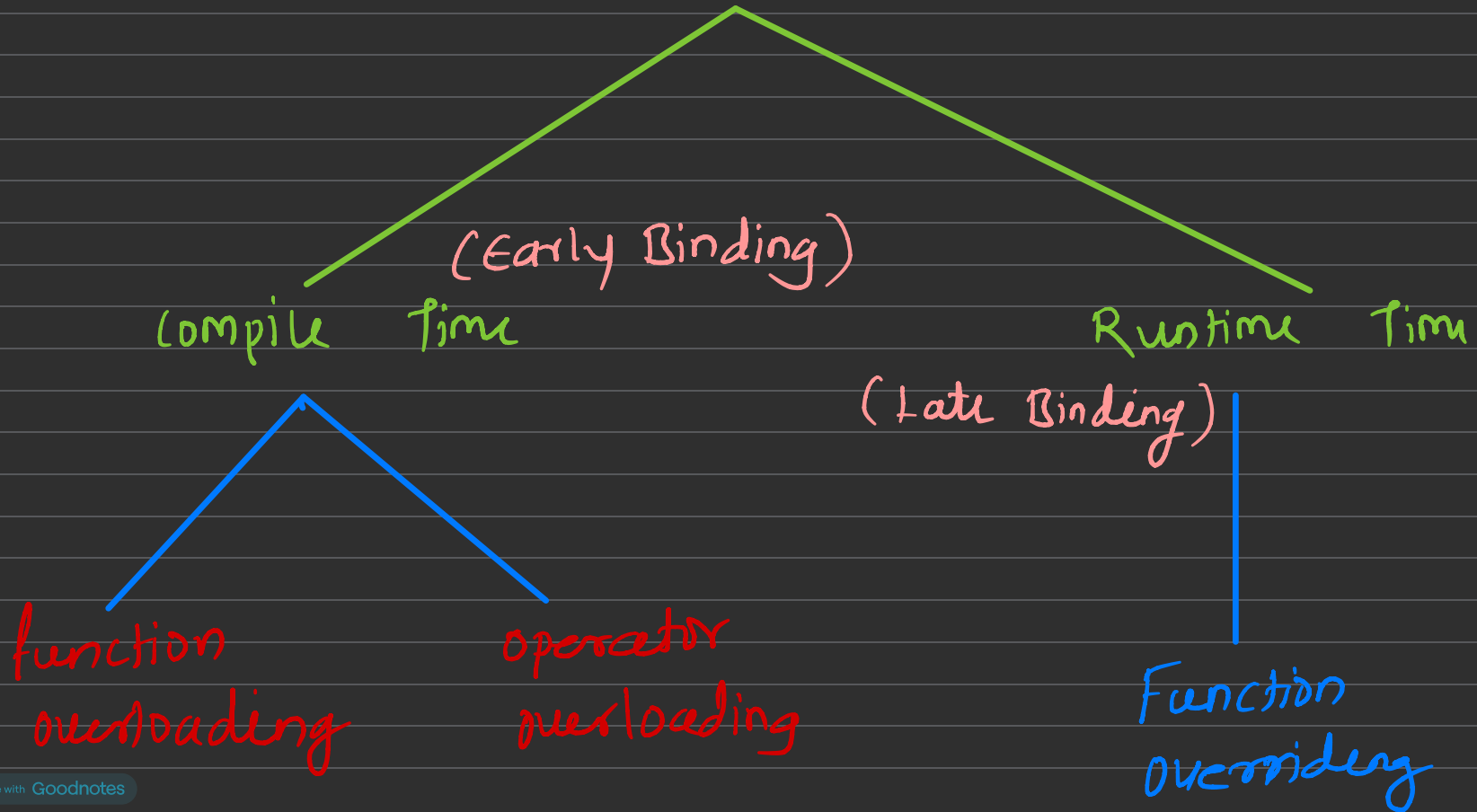
```
class Sushant ==
```



Polymorphism:



Types of polymorphism



Compile time: (function overloading)

Sum 1

int x = 10

int y = 20

int z = x + y

Sum 2

float x = 10.0

float y = 40.0

float z = x + y

Sum 3

char c = 'q'

char d = 'o'

char z = c + d

- all functions should be in same class

⊖ Return type does not matter

⊖ function should have same name but different parameters

```
class Demo
```

```
{
```

```
    void add (int x, int y)
```

```
    {  
        print (x+y)
```

```
    }
```

```
    int add ( int x, float y)
```

```
    {  
        return (x+y)
```

```
    }
```

```
}
```

```
Demo obj
```

```
obj.add (10, 20)
```

```
obj.add (10, 30.4)
```

function Override (Runtime / Late Binding)

- function must have same Name, same param. and same return type
- functions should be in diff. class

```
int x = 10  
x = 40
```

x
1040

```
class Person
```

```
{ public:
```

```
    void info()
```

```
    { cout << " This is Person info"
```

```
    }
```

```
}
```

```
class Yash : public Person
```

```
{ public:
```

```
    void info()
```

```
    { cout << " This is Yash info"
```

```
    }
```

```
}
```

Yash obj

obj.info()

Operator Overloading

```
class A  
{  
    int x  
    A() { x=40;  
}
```

A obj1

A obj2

A ans = obj1 + obj2

int x = 10

int y = 40

z = x + y

int x = 100

float z = 40.8

ans = x + z

NOTE

— Arithmetic operators (+, -, *, /)
can be applied only on
Primitive Data Type

Keyword

function
Name

obj1 + obj2

parameter of function

Name of function

(current object

whose values get passed

as parameter to constructor

void operator + (className obj2)

```

class Add
{
    int x, y;

    public:
        Add ( int x, int y )
        {
            this → x = x;
            this → y = y;
        }

        void operator + ( Add obj )
        {
            X = x + obj.x;
            Y = y + obj.y;
            cout << x << Y;
        }
}

```

Add obj₁(10, 30)

Add obj₂(40, 80)

obj₁ + obj₂